



A. Problem Description and Objective:

Consider a scenario where two end users namely, A and B, desire to communicate and exchange data messages over a packet switching network using UDP. This latter protocol known to be unreliable with absolutely no mechanisms allowing for recovery from events such as packet loss, delayed packet delivery, out of order delivery and erroneous packet reception, it becomes highly likely that A and B shall experience bad communication performance as a result of the occurrences of such events. This is especially true when it comes to operating over a network with a large number of communicating users, high traffic intensity and significant data volumes plying back and forth over the network's links bringing them to almost exhausted capacity. However, when developing network applications within the frame of a simple project that is tested over very small networks and often over a virtual network environment hosted by a single machine, the above-mentioned reliability-related challenges are pretty unlikely to occur. For this reason, the purpose of the first phase of this semester's project is to develop a UDP server that has the objective of simulating random packet losses and packet delivery delays.

Precisely, it is required to develop a JAVA-based channel simulation application called, `UnreliableChannel.java`, whose objective is to simulate randomized packet losses and delayed packet deliveries. To do so, you have to imagine the scenario of two communicating end hosts exchanging messages over an unreliable channel where delays and losses are likely to occur as depicted in Figure 1(a). Then, as in Figure 2(b), the channel shall be replaced with a node, typically a server say S, where the `UnreliableChannel.java` application shall be hosted and executed. Consequently, when user A would like to communicate with user B, it'll send the message to the server S. This latter will decide, with a certain probability p , to discard the received message from A or, with a probability of $1 - p$, to relay this message to user B after some random delay, say D .



(a) An imagination of the communication scenario between A and B over an unreliable channel.



(b) The communication scenario between A and B through a server simulating the unreliable channel.

Figure 1: Communication among two end-hosts over an unreliable channel.

B. Basic Requirements:

Packets must be transmitted from user A to user B and vice-versa (appropriate client applications must be designed to generate and perform such transmissions). Consider the operation of one user, say A (the other user, B, operates similarly). It is required to enable A to continuously generate packets in a periodic manner at a rate of one packet every half a second. Currently, for simplicity, it is going to be assumed that all generated packets have equal sizes containing a very simple message consisting of the source user's name and destination user's name (*i.e.*, A or B) separated by a white space then another white space followed by the packet's sequence number (*i.e.*, an integer being either 0 or 1 flipping from one packet to the other). Each user must transmit a total number of one thousand packets and must maintain a count of the received packets from the other user. Upon the transmission of the required number of packets, each user must transmit an end of session signal (*i.e.*, a packet containing only the String "END" without the double quotes) to the channel simulation server S.

Now, let's move to the simulation server's side. The server S must listen on a user-specified port for incoming UDP packets. It must receive the packet and then introduce controlled randomness to simulate packet loss and delay. One way to simulate packet losses is to define a real constant probabilistic loss rate factor, say $p = 0.3$ (*i.e.*, 30% of the arriving packets to S will be immediately discarded, in other words, lost). Then, a random generator, say `rnd`, must be used to generate a random real number, say `d`, within the range $[0; 1]$. If `d` is smaller than or equal to p , then the received packet is immediately dropped and the server must then listen to the given port for the next incoming packet. Otherwise, the packet shall be assumed to be reaching the destination user but with some delay. One way to simulate the delay is to define a delay range bounded by certain minimum delay, say `minD` (typically 0), and maximum delay, say `maxD` (*e.g.*, 200 ms). Then, use `rnd` to generate a uniformly distributed random integer delay, say `D`, within the range $[\text{minD}; \text{maxD}]$. Consequently, the server shall be forced to sleep for an amount of time that is equal to `D`. Once it wakes up, the server must transmit the received packet from the source user to its intended destination user. The network operator must be allowed to control the severity of unreliability through a permission to provide the parameters `p`, `minD` and `maxD` values as command-line arguments specified upon launching the `UnreliableChannel.java` application on the server. In terms of statistics, the server must keep track of the total number of packets received from each user, how many of each user's packets were dropped, how many were delayed and, so, how many have been successfully delivered. In addition the server must display the average packet delivery delay in each communication direction (*i.e.*, from A to B and vice-versa). Such statistics must be clearly displayed to the screen upon the termination of the session between the communicating users (*i.e.*, once both users declare the end of the data exchange as mentioned above). A sample output of the server is as follows:

```
Packets received from user A: 1000 | Lost: 275 | Delayed: 725
Packets received from user B: 1000 | Lost: 325 | Delayed: 675
Average delay from A to B: 135.8 ms.
Average delay from B to A: 115.4 ms.
```

C. Additional Required Features:

1. Variable sized packets with different messages per packet. The content of each packet must then be displayed at the server's side together with the IP address of the packet's sender and receiver.
2. Implementation of different packet loss patterns (*e.g.*, bursts) for more realistic simulations.
3. Allow concurrent connections of multiple users. More creative delay and loss mechanisms might be required here. The use of multiple server threads might come in handy. Creativity at this point shall be highly awarded.
4. Offer options to vary delay distributions (*e.g.*, Gaussian, exponential, etc).
5. Offer options to vary the packet generate distributions (*e.g.*, Poisson).
6. Handle potential errors during socket creation, connection establishment and data processing.
7. Provide the user with informative error messages in the form of logs for debugging purposes.

D. Resources:

- Java socket programming documentation:
<https://docs.oracle.com/javase/tutorial/networking/sockets/index.html>
- Random number generation in Java:
https://docs.oracle.com/cd/E17802_01/j2se/j2se/1.5.0/jcp/beta1/apidiffs/java/util/Random.html
- Implementing sleep in Java:
<https://docs.oracle.com/javase/tutorial/essential/concurrency/sleep.html>
- Command-line arguments in Java:
<https://www.tutorialspoint.com/Command-Line-arguments-in-Java-programming>